

Inter Thread Communication in C# with Examples

In this article, I am going to discuss **Inter Thread Communication in C#** with Examples. Please read our previous article where we discussed [How to Terminate a Thread in C#](#) with Examples.

Interthread Communication in C#:

Interthread Communication in C# is a mechanism of communication between two or more threads that work on the shared resource. To perform multiple actions at the same time, we need Inter-thread communication. In order to have smooth interthread communication between multiple threads in C#, we can use the Monitor Class. The Monitor class in C# provides some static methods such as `Wait()`, `Pulse()`, and `PulseAll()` which we can use to perform Interthread Communication between multiple threads. The Monitor class belongs to `System.Threading` namespace.

If you want to learn and understand the C# Monitor Class in detail, then please click on the below URL.

<https://dotnettutorials.net/lesson/multithreading-using-monitor/>

Methods of Monitor Class for Interthread communication in C#:

1. **`public static bool Wait(Object obj)`**: This method releases the lock on an object and blocks the current thread until it reacquires the lock. In other words, this method makes the thread that has called the `Wait()` method wait for the other thread to complete its work on the same object. Here, the parameter `obj` specifies the object on which to wait. It returns true if the call is returned because the caller reacquired the lock for the specified object. This method does not return if the lock is not reacquired. It will throw `ArgumentNullException` if the `obj` parameter is null. It will throw `SynchronizationLockException` if the calling thread does not own the lock for the specified object. And this method will throw `ThreadInterruptedException` if the thread that invokes `Wait` is later interrupted from the waiting state. This happens when another thread calls this thread's `System.Threading.Thread.Interrupt` method.
2. **`public static void Pulse(object obj)`**: This method notifies a thread in the waiting queue of a change in the locked object's state. That means The thread was waiting(after calling `Wait()` method) on the same object. Here, the parameter `obj` specifies the object a thread is waiting for. It will throw `ArgumentNullException` if the `obj` parameter is null. It will throw `SynchronizationLockException` if the calling thread does not own the lock for the specified object.
3. **`public static void PulseAll(object obj)`**: This method notifies all waiting threads of a change in the object's state i.e. about the release of lock over the object. Here, the parameter `obj` specifies the object that sends the pulse. It will throw `ArgumentNullException` if the `obj` parameter is null. It will throw `SynchronizationLockException` if the calling thread does not own the lock for the specified object.

Note: The Calling of `Wait()`, `Pulse()`, `PulseAll()` method is only possible from within the synchronized context i.e. from within a synchronized block with a lock.

Example to Understand Interthread Communication in C#:

The `Wait()` Method of Monitor Class is used to Release the lock on an object in order to permit other threads to lock and access the object. The calling thread waits while another thread accesses the object. The `Pulse` signals are used to notify waiting threads about changes to an object's state. For a better understanding, please have a look at the below example.

Let us understand this with a real-time example. Our business requirement is to print the Even and Odd number sequence using 2 different threads. So, One thread will print the even numbers and another thread will print the odd numbers.

Thread T1: 0,2,4,6,8...

Thread T2: 1,3,5,7,9...

Output: 0,1,2,3,4,5,6,7,8,9...

To solve the problem let's use the signaling mechanism using the Monitor Class Wait() and Pulse() Methods in C#. In the following example, we use the Monitor.Wait() method to make the thread waiting and Monitor.Pulse() method to signal other thread. The process is as follows:

1. First, the Even thread will start to print the number on the console.
2. Then the Even thread will signal the Odd thread to print the number using the Monitor.Pulse() method.
3. Again, the Even thread will wait for a signal from the Odd thread using Monitor.Wait() method.
4. The same thing will also be done by the Odd Thread.
5. The Odd thread will start to print the number on the console.
6. Then the Odd thread will signal the Even thread to print the number using Monitor.Pulse() method.
7. Again, the Odd thread will wait for the signal from the Even thread using Monitor.Wait() method.
8. The same process is going on.

Since both the Odd and Even threads are sharing the same console window to print the number, we need to put a lock on the console IO. We want the sequence to be started with the even number, So, the even thread must run first. Once, we will start the Even thread, then we need to pause for a moment before starting the Odd thread using the Sleep() method of the Thread class in C# to avoid any chance to start the Odd thread first.

```
using System;
using System.Threading;
namespace InterthreadCommunications
{
    class Program
    {
        //Limit numbers will be printed on the Console
        const int numberLimit = 10;
        static readonly object _lockObject = new object();
        static void Main(string[] args)
        {
            Thread EvenThread = new Thread(PrintEvenNumbers);
            Thread OddThread = new Thread(PrintOddNumbers);
            //First Start the Even thread.
            EvenThread.Start();
            //Pause for 10 ms, to make sure Even thread has started
            //or else Odd thread may start first resulting different sequence.
            Thread.Sleep(100);
            //Next, Start the Odd thread.
            OddThread.Start();
            //Wait for all the childs threads to complete
            OddThread.Join();
            EvenThread.Join();
            Console.ReadKey();
        }
    }
}
```

```

}
//Printing of Even Numbers Function
static void PrintEvenNumbers()
{
    try
    {
        //Implement lock as the Console is shared between two threads
        Monitor.Enter(_lockObject);
        for (int i = 0; i <= numberLimit; i = i + 2)
        {
            //Printing Even Number on Console)
            Console.Write($"{i} ");
            //Notify Odd thread that I'm done, you do your job
            Monitor.Pulse(_lockObject);
            //I will wait here till Odd thread notify me
            // Monitor.Wait(monitor);
            //Without this logic application will wait forever
            bool isLast = false;
            if (i == numberLimit)
            {
                isLast = true;
            }
            if (!isLast)
            {
                //I will wait here till Odd thread notify me
                Monitor.Wait(_lockObject);
            }
        }
    }
    finally
    {
        //Release the lock
        Monitor.Exit(_lockObject);
    }
}

//Printing of Odd Numbers Function
static void PrintOddNumbers()
{
    try
    {
        //Hold lock as the Console is shared between two threads
        Monitor.Enter(_lockObject);
        for (int i = 1; i <= numberLimit; i = i + 2)
        {
            //Printing the odd numbers on the console
            Console.Write($"{i} ");
            //Notify Even thread that I'm done, you do your job
            Monitor.Pulse(_lockObject);
            //I will wait here till even thread notify me
            // Monitor.Wait(monitor);
            // without this logic application will wait forever

```

```

        bool isLast = false;
        if (i == numberLimit - 1)
        {
            isLast = true;
        }
        if (!isLast)
        {
            //I will wait here till Even thread notify me
            Monitor.Wait(_lockObject);
        }
    }
}
finally
{
    //Release lock
    Monitor.Exit(_lockObject);
}
}
}
}

```

Output: 0 1 2 3 4 5 6 7 8 9 10

How do Wait() and Pulse() methods help to Implement Interthread Communication in C#?

Let us see some more examples to understand the importance of Wait() and Pulse() methods in order to implement the Interthread Communication in C#. Suppose we have two threads i.e. Thread1 and Thread2, where, Thread1 has to print the table of 4, and Thread2 has to print the table of 5 and the table of 4 should be printed before the table of 5, in order to keep an ascending order of tables.

By using Wait() and Pulse() methods for inter-thread communication, Thread2 will call the Wait() method, to wait for Thread1 to finish printing the table of 4. On completing its task, the Thread1 will call Pulse() method, which notifies Thread2 to continue its work of printing the table of 5.

Without the use of Wait() and Pulse() methods, when both threads start at the same time, Thread2 may print the table of 5, before Thread1 has printed the table of 4. Let's see code examples proving each of these cases.

Example without using the Wait and Pulse method in C#:

In the below example, We are going to have two threads. The first thread i.e. Main Thread is given to us by default. And then We manually create a new thread. The manually created new thread will print a table of 4 and the Main Thread will print a table of 5. But, without using the Wait() and Pulse() methods, when two threads start at almost the same time, The Main Thread may print the table of 5 before the manually created thread has printed the table of 4. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;

```

```

namespace InterthreadCommunications
{
    class Program
    {
        static readonly object _lockObject = new object();
        static void Main(string[] args)
        {
            //Creating an object of Thread class to Execute the PrintTable method
            Thread thread = new Thread(PrintTable)
            {
                Name = "Manual Thread"
            };
            thread.Start();
            //Locking the _lockObject
            lock (_lockObject)
            {
                Thread th = Thread.CurrentThread;
                th.Name = "Main Thread";
                Console.WriteLine($"{th.Name} Running and Printing the Table of 5");
                for (int i = 1; i <= 10; i++)
                {
                    Console.WriteLine("5 x " + i + " = " + (5 * i));
                }
            } //synchronized block ends
            Console.ReadKey();
        }

        public static void PrintTable()
        {
            //Synchronizing or locking the _lockObject
            //Doing so, restricts any other thread to access a block of code using
            _lockObject at the same time.
            lock (_lockObject)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} Running and Printing the
                Table of 4");
                for (int i = 1; i <= 10; i++)
                {
                    Console.WriteLine("4 x " + i + " = " + (4 * i));
                }
            }
        }
    }
}

```

Output:

```

Main Thread Running and Printing the Table of 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
Manual Thread Running and Printing the Table of 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40

```

In the above example, the Main thread enters the synchronized block, gets the lock on the `_lockObject`, and gets its synchronized sole access to print the table of 5. It then releases the lock on the `_lockObject`.

In the `PrintTable` method, Manual Thread enters the synchronized block and gets the lock on the same `_lockObject`. It then prints the table of 4. Hence, table 5 is printed before table 4, bothering our ascending order, because there was no interthread communication.

Interthread Communication Example using `Wait()` and `Pulse()` Methods in C#

We are going to have two threads in the below example. The first thread i.e. Main Thread is given to us by default. And we will manually create a new thread, based on an object of the class. The manually created new thread will print a table of 4 and the Main Thread will print a table of 5. We will use `Wait()` and `Pulse()` methods for communication between these two threads, in such a way that table 4 is printed before table 5, to maintain an ascending order.

```

using System;
using System.Threading;
namespace InterthreadCommunications
{
    class Program
    {
        static readonly object _lockObject = new object();
        static void Main(string[] args)
        {
            //Creating an object of Thread class to Execute the PrintTable method

```

```

Thread thread = new Thread(PrintTable)
{
    Name = "Manual Thread"
};
thread.Start();
//Locking the _lockObject
lock (_lockObject)
{
    //Calling the Wait() method in a synchronized context
    //Doing so, makes the Main Thread stops its execution and wait
    //until it is notified by the Pulse() method
    //on the same object _lockObject
    Monitor.Wait(_lockObject);
    Thread th = Thread.CurrentThread;
    th.Name = "Main Thread";
    Console.WriteLine($"{th.Name} Running and Printing the Table of 5");
    for (int i = 1; i <= 10; i++)
    {
        Console.WriteLine("5 x " + i + " = " + (5 * i));
    }
} //synchronized block ends
Console.ReadKey();
}
//The entry-point method of the thread
public static void PrintTable()
{
    //Synchronizing or locking the _lockObject
    //Doing so, restricts any other thread to access a block of code u
    _lockObject at the same time.
    lock (_lockObject)
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} Running and Pr
        Table of 4");
        for (int i = 1; i <= 10; i++)
        {
            Console.WriteLine("4 x " + i + " = " + (4 * i));
        }
        //The manually created thread is calling the Pulse() method
        //To notifying the Main thread that it is releasing the lock c
        _lockObject
        //And Main Thread could lock the object to continue its work
        Monitor.Pulse(_lockObject);
    } //synchronized block ends
}
}
}

```

Output:

```
Manual Thread Running and Printing the Table of 4
```

```
4 x 1 = 4  
4 x 2 = 8  
4 x 3 = 12  
4 x 4 = 16  
4 x 5 = 20  
4 x 6 = 24  
4 x 7 = 28  
4 x 8 = 32  
4 x 9 = 36  
4 x 10 = 40
```

```
Main Thread Running and Printing the Table of 5
```

```
5 x 1 = 5  
5 x 2 = 10  
5 x 3 = 15  
5 x 4 = 20  
5 x 5 = 25  
5 x 6 = 30  
5 x 7 = 35  
5 x 8 = 40  
5 x 9 = 45  
5 x 10 = 50
```

In the above code, we manually created a thread called Manual Thread. The Main thread enters the synchronized block, gets the lock on the `_lockObject`, and gets its synchronized sole access to print the table of 5. However, the Main Thread calls the `Wait()` method, doing so, it releases the lock on the `_lockObject` and stops its execution.

In the `PrintTable` method, Manual Thread enters the synchronized block and gets the lock on the same `_lockObject`. It then prints the table of 4 and once it prints the table of 4, then it calls the `Pulse()` method to notify the waiting Main Thread and releases the lock on `_lockObject`. On being notified, the Main Thread wakes up, locks the `_lockObject`, and completes its execution by printing the table of 5.

Note: In our example, we call the `Wait()` or `Pulse()` methods from within a synchronized context, otherwise such interthread communication would not have been possible.

In the next article, I am going to discuss [How to Debug a Multi-threaded Application in C#](#) with Examples. Here, in this article, I try to explain **Inter Thread Communication in C#** with Examples. I hope you enjoy this **Inter Thread Communication in C# with Examples** article.